2.GANs

04 February 2026 12:53

framework → 2 Models → Generator, Discriminator } Adversarial process

Generator → tries to capture data distribution

Discriminator → tries to critique if a sample received is from trainingset / G

The training process/Aim is to (maximize) the probability of 'D' making a mistake

in case of (random functions / arbitrary / simple) there is a possibility where G might recover training data with probability becoming 1/2

Adversarial Networks framework

Noise contrastive Estimation → Uses a fixed Noise distribution

(in this paper rather than a separate discriminative model → genererative model is used to discriminate generated data from samples

→ learning slows dramatically after the model has learned even an approximately correct distribution over a small set of observed variables

Adv Nets → both models are MLP's (neural nets)

To learn generators distribution 'p_g' over data 'x' we define a prior on input noise variables $p_z(z)$ → The paper is simply saying that we start with a simple random noise z sampled from a known distribution → we then transform that noise using a neural network G to produce data samples

↓ is basically a prior distribution over Latent variables z (simply put → random source we sample from)

prior in general means → A probability distribution we choose before viewing any data } represents our initial assumptions

Represent mapping to data space as $G(z, \theta_g)$ → parameters

↓ differentiable function represented by MLP

$D(x, \theta_d)$ → second MLP → outputs a single scalar

D(x) represents prob that 'x' came from data rather than 'p_g'

we train D to maximize the probability of assigning the correct label

we simultaneously train 'G' to (minimize $\log(1 - D(G(z)))$)

D(x) → probability that sample is real

$1 - D(G(z))$ → probability that sample is fake

$D(G(z)) \to 1$ Generator winning

$D(G(z)) \to 0$ Discriminator winning

paper objective → Maximize correct classification

→ Make D(x) close to 1 for real data

D(G(z)) close to zero for generated data

Screenshot _2026-02-...

Alternating between k steps of optimizing D and one step of G

* the above min $\log(1 - D(G(z)))$ causes vanishing gradients early in training } same ideas but modern one has better gradients

Modern GANs use max $\log(D(G(z)))$ called non-saturating form

Why do we use log? Because this is a log-likelihood / cross entropy objective

↓ a model predicts probabilities we maximize it $P \to \log(P)$

↓ log converts multiplication to Addition & makes optimization easier

↓ Strong gradients when wrong

$\log(P)$ → Near 1 → small changes matter less

Near 0 → large penalty } punishing confident mistakes

D & G play the following min max game with the value function V(G, D)

probability → parameters known → predict outcome

likelihood → output known → estimate the parameters

Max likelihood Estimation (MLE)

Goal → You have a data set & a model

Process → You adjust weights to find values that maximizes the likelihood of our data occuring

Result → when you train a model you are essentially searching for parameters that makes observed data as unsurprising as possible

$$\min_G \max_D V(G, D) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))]$$

↓ Objective game b/w generator & discriminator

↓ probability that real data is real

↓ probability that generated data is real

Objective game b/w generator & discriminator

probability that real data is real

probability that generated data is real

D tries to maximize value function & G tries to minimize it

E simply means Expectation (in simpler terms Average value)

$\mathbb{E}_{x \sim p_{data}(x)}[\log(D(x))] \rightarrow$ avg of $\log D(x)$ where x is sampled from real data distribution

x is sampled from real data distribution

optimizing D to completion in the inner loops of training is computationally prohibitive

on finite datasets → overfitting

basically this says training D fully until becomes almost perfect before updating G

{ problematic → D would need more steps every iteration

We alternate b/w 'k' steps of optimizing D & one step of optimizing G

so long as G changes slowly enough } each G update → small so that distribution produced by 'G' doesn't suddenly jump

Result → when you train a model you are essentially searching for parameters that makes observed data as unsurprising as possible

→ Earlier in training { if according to value eqn from above }

Generator provides samples that are completely different to sample data

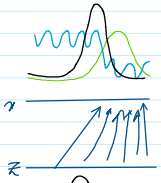
the D rejects samples with high confidence G is poor the gradient becomes saturated $\log(1-D(G(z)))$

so training is very slow

Instead of training G to minimize $\log[1-D(G(z))]$

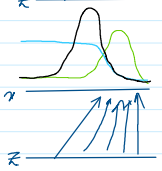
we train it to maximize $\log D(G(z))$

(a)



okay to remember GAN's trained by simultaneously updating discriminative distribution (blue line) so that it discriminates between samples from data generating distribution (black line) P_x from those of the generative distribution P_g (green line)

(b)

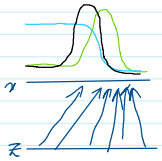


Lower horizontal line (domain from which z is sampled)

upper horizontal line (domain of x)

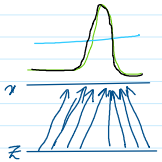
The arrows show how mapping $x = G(z)$ imposes P_g on transformed samples

(c)



'G' contracts in regions of high density of P_g & vice versa

(d)



(a) adversarial pair near convergence $P_g \approx P_{data}$ D is partially accurate classifier

(b) Inner loop of algorithm D → trained to discriminate samples from data converging to $D^*(x) = \frac{P_{data}(x)}{P_{data}(x) + P_g(x)}$

(c) After an update to G, gradient of D has guided G(z) to flow to regions that are more likely classified as data

(d) After several steps, if D & G have enough capacity the reach a state where both can't improve $P_g \approx P_{data}$ $D(x) = \frac{1}{2}$ Discriminator is unable to differentiate

Start the result

(d) After several steps, if D & G have enough capacity the reach a state where both can't improve $P_g \approx P_{data}$ $D(x) = \frac{1}{2}$ Discriminator is unable to differentiate

Theoretical results

Normally G & D are neural nets with finite parameters, but here the assume these to have infinite capacity → { can represent any possible function; No architectural milestones; No parametric constraints }

SGD takes one sample instead of full dataset for iteration like GD

They study convergence in space of probability density functions (instead of adjusting weights)

Mini Batch / Stochastic Gradient descent training of GAN's

core components → GAN's
→ SGD
→ Mini Batch

Algorithm 1 MBSGD training — no of steps for discriminator 'k' is a hyperparameter

They used k=1

for no of training iterations do

for k steps do

Sample mini batch of m noise samples $\{z^{(1)} \ldots z^{(m)}\}$ from noise prior $P_g(z)$

Sample " " m examples $\{x^{(1)} \ldots x^{(m)}\}$ in data generating distribution $P_{data}(x)$

Update the discriminator by ascending its stochastic gradient

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D(x^{(i)}) + \log\left(1 - D(G(z^{(i)}))\right) \right]$$

→ has two terms because D must classify real & fake samples so its loss contains both $\log D(x)$ & $\log(1 - D(G(z)))$ (real term, fake term)

end for

sample minibatch of m noise samples $\{z^{(1)} \ldots z^{(m)}\}$ from noise prior $P_g(z)$

update the generator by descending its stochastic gradient

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D(G(z^{(i)}))\right)$$

→ G only cares about how fake samples are judged so only one term

end for

They used momentum but any gradient based learning rule can be used

Global optimality for $P_g = P_{data}$

They consider optimal discriminator D for any given generator G

proposition 1 For G fixed, the optimal discriminator D is

$$D^*_G = \frac{P_{data}(x)}{P_{data}(x) + P_g(x)}$$

The training criterion for D for any given G is to maximize $V(G, D)$

$$V(G, D) = \int_x P_{data}(x) \log(D(x))\,dx + \int_z P_z(z) \log(1 - D(g(z)))\,dz$$

$$= \int_x P_{data}(x) \log(D(x)) + P_g(x) \log(1 - D(x))\,dx$$

Expectation over distribution is an integral

$E_{x \sim p(x)} f(x) = \int p(x) f(x)\,dx$

$x = G(z)$ induces distribution $P_g(x)$ the randomness of z gets pushed through G

z is gaussian G stretches & bends space the output cloud of points has some shape

once they move everything to data space x

for each fixed x the objective looks like $a \log y + b \log(1-y)$

$a = P_{data}(x)$ $y = 1$ (real) $y = 0$ fake

$b = P_g(x)$

$y = D(x)$ $y = \frac{a}{a+b}$

for any $(a, b) \in \mathbb{R}^2 \setminus \{0, 0\}$ the function $y \to a \log(y) + b \log(1-y)$ achieves its maximum in $[0, 1]$ at $a/a+b$

The discriminator doesn't need to be defined outside of $Supp(P_{data}) \cup Supp(P_g)$

The training criterion for D can be interpreted as maximizing the log likelihood for estimating

for any $(a,b) \in \mathbb{R}^2 \setminus \{0,0\}$ the function $y \to a\log(y) + b\log(1-y)$ achieves its maximum on $[0,1]$ at $a/a+b$

The discriminator doesn't need to be defined outside of $Supp(P_{data}) \cup Supp(P_g)$

The training objective for D can be interpreted as maximizing the log likelihood for estimating the conditional probability $P(Y=y|x)$ Y indicates whether x comes from P_{data} (with $y=1$) or from P_g (with $y=0$)

The minmax from eq(1) can be rewritten as

$$C(G) = \max_D V(G,D)$$
$$= E_{x\sim P_{data}}\left[\log D^*_G(x)\right] + E_{z\sim P_z}\left[\log(1-D^*_G(G(z)))\right]$$
$$= E_{x\sim P_{data}}\left[\log D^*_G(x)\right] + E_{x\sim P_g}\left[\log(1-D^*_G(x))\right]$$
$$= E_{x\sim P_{data}}\left[\log \frac{P_{data}(x)}{P_{data}(x)+P_g(x)}\right] + E_{x\sim P_g}\left[\log \frac{P_g(x)}{P_{data}(x)+P_g(x)}\right]$$

Maximize objective → gradient ascent
Minimize objective → gradient descent

Original term $E_{z\to P_z}(\log(1-D(G(z))))$
But since $x = G(z)$ those x's follow distribution $P_g(x)$
hence → $E_{x\to P_g}[\log(1-D(x))]$
Expectation equals integral
$\int P_g(x)\log[1-D(x)]$
Basically, if you sample from z, transform it through G (since $x=G(z)$), the outputs follow distribution P_g

first term in X-space
Second in z-space
This is messy so we rewrite second in X-space
hence both terms are functions of x
integrated over the same domain

Theorem 1 — The global minimum of the virtual training criterion $C(G)$ is achieved if & only if $P_g = P_{data}$. At that point $C(G)$ achieves the value $-\log 4$

Proof — for $P_g = P_{data}$, $D^*_G(x) = 1/2$

Hence the equation → $C(G) = \max V(G,D) = E_{x\sim P_{data}}\left[\log \frac{P_{data}(x)}{P_{data}(x)+P_g(x)}\right] + E_{x\sim P_g}\left[\log \frac{P_g(x)}{P_{data}(x)+P_g(x)}\right]$

→ We find $C(G) = \log 1/2 + \log 1/2 = -\log 4$

$E_{x\to P_{data}}[-\log 2] + E_{x\to P_g}[-\log 2] = -\log 4$ To see that this is the best value of $C(G)$, reached only for $P_g = P_{data}$

$$C(G) = V(D^*_G, G)$$
$$C(G) = -\log(4) + KL\left(P_{data} \,\middle\|\, \frac{P_{data}+P_g}{2}\right) + KL\left(P_g \,\middle\|\, \frac{P_{data}+P_g}{2}\right)$$

Kullback Leibler divergence

We recognize in the previous expression the Jensen Shannon divergence between the models distribution and the data generating process

$$C(G) = -\log(4) + 2\,JSD(P_{data}\|P_g)$$

between two distributions is always non negative and zero only when they are equal

Hence $C^* = -\log(4)$ is the global minimum of $C(G)$ and the only solution is $P_g = P_{data}$ the generative model perfectly replicating the data generating process

KL divergence (only what we need)
↳ measures how different one distribution is from another
Notation $KL(P\|Q)$
{ Means how much information is lost if Q is used to represent P }
Key properties (only important ones)
$KL(P\|Q) \geq 0$
equals 0 only when $P=Q$ } only property theorem 1 needs

JSD (Jensen-Shannon divergence)
JSD is a symmetric version of KL
Take average distr^n $M = \frac{P+Q}{2}$
Then $JSD(P\|Q) = \frac{1}{2}KL(P\|M) + \frac{1}{2}KL(Q\|M)$
KL alone is asymmetric, JSD is symmetric & stable

* $JSD(P\|Q) \geq 0$
and $JSD(P\|Q) = 0 \iff P = Q$

$KL(P\|Q) \geq 0$
equals 0 only when $P=Q$ } only property theorem 1 needs

intuition → if distributions are identical → difference = 0
if different → difference > 0

GAN tries to reduce JSD between real & generated distr.

KL alone is asymmetric, JSD is symmetric & stable

* $JSD(P\|Q) \geq 0$
only $JSD(P\|Q) = 0 \iff P = Q$

here $C(G)$ is rewritten as

$$-\log 4 + KL(P_{data} \| M) + KL(P_g \| M) \qquad M = \frac{P_{data} + P_g}{2}$$

$$\Rightarrow -\log 4 + JSD(P_{data} \| P_g)$$

Now its obvious since $JSD \geq 0$

$$C(G) \geq -\log 4$$

Minimum occurs when $JSD = 0$ which happens only if $P_g = P_{data}$

Convergence of Algorithm 1

proposition 2 - if G & D have enough capacity
& at each step of Algo 1 the D is allowed to reach its optimum given G, P_g is updated so as to improve the criterion

$$C(G) = \mathbb{E}_{x \sim P_{data}}\left[\log D^*_G(x)\right] + \mathbb{E}_{x \sim P_g}\left[\log(1 - D^*_G(x))\right]$$

then P_g converges to P_{data}

Where does two terms come from
they rearrang eq 4

We have

$$C(G) = \mathbb{E}_{x \sim P_{data}}\left[\log \frac{P_{data}(x)}{P_{data}(x) + P_g(x)}\right] + \mathbb{E}_{x \sim P_g}\left[\log \frac{P_g(x)}{P_{data}(x) + P_g(x)}\right]$$

write them as integrals

$$C(G) = \int P_{data}(x) \log \frac{P_{data}(x)}{P_g(x) + P_{data}(x)} + \int P_g(x) \log \frac{P_g(x)}{P_{data}(x) + P_g(x)}$$

proof - consider $V(G,D) = U(P_g, D)$ as a function of P_g
Note that $U(P_g, D)$ is convex in P_g

The subderivatives of supremum of convex functions includes derivative of the function at point where maximum is attained

intuitive translation would be
if you always pick a best discriminator updating G is still like doing gradient descent on a convex objective

it simply says if D is always optimized (but possible for current G) then updating G is equivalent to doing gradient descent on a convex function

if $f(x) = \sup_{\alpha \in A} f_\alpha(x)$ & $f_\alpha(x)$ is convex in x for every α
then $\partial f_\beta(x) \in \partial f$ if $\beta = \arg\sup_{\alpha \in A} f_\alpha(x)$

This is equivalent to computing a gradient descent update for P_g at the optimal D given corresponding G

$\sup_D U(P_g, D)$ is convex in P_g with a unique global optima as proven in theorem 1

Therefore with sufficiently small updates of P_g, P_g converges to P_x

supremum 'sup' means upperbound

$f(x) = \sup_{\alpha \in A} f_\alpha(x)$ means take many function $f_\alpha(x)$
pick the largest value among them

just indexing many functions → imagine many curves

for each x → look at all curves take the highest one → This creates a new function → that's the supremum

Why supremum is mentioned because D is maximized

when they write $\sup_D U(P_g, D)$ they mean objective after choosing best possible discriminator

Notes KL divergence

(continuous case) $KL(P\|Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$

→ data is assumed to come from P
but we use Q to represent it
it measures the penalty for that mismatch

Imagine P = reality & Q = our model
→ how wrong our model is at reality follows P

KL can explode when
$Q(x) = 0$ but $P(x) > 0$
because in $\int P(x) \log\left(\frac{P(x)}{Q(x)}\right)$ $\log(P/Q) \to \infty$

Imagine P = reality & Q = our model

→ how wrong your model is if reality follows P

KL is asymmetric meaning → $KL(P\|Q) \neq KL(Q\|P)$

why? $P(x)\log\frac{P(x)}{Q(x)}$ → weighting comes from P & not Q

$KL(P\|Q)$ emphasizes where P is large & vice versa for $KL(Q\|P)$

JSD → symmetric?

let's say P has mass somewhere Q has not

P/Q → very large → huge penalty → reversing them may not give same result

$JSD(P\|Q) = \frac{1}{2}KL(P\|M) + \frac{1}{2}KL(Q\|M)$

$M = \frac{P+Q}{2}$

We treat P & Q equally & compare to the average M, Both weighed equally

so $JSD(P\|Q) = JSD(Q\|P)$

Why JSD is related to KL

↳ it simply a combination of two KL divergences

(Averaged KL distance to midpoint distribution)

Why introduce the avg distribution M

because KL compares directly

JSD says instead

find middle ground

Measure how far P & Q are

Average them

creating symmetry & stability

$Q(x)=0$ but $P(x)>0$

because in $\int P(x)\log\left(\frac{P(x)}{Q(x)}\right)$ $\log(P/Q) \to \infty$

JSD avoids this because M contains both (disappears)

hence stable

$C(G) = \int P_{data}\log\frac{P_{data}}{M}dx + \int P_g\log\frac{P_g}{M}dx - \log 2\int P_{data}(x)dx - \log 2\int P_g(x)dx$

Since distrs integrate to 1 $\int P_{data}(x)dx=1$ $\int P_g(x)dx=1$

$\Rightarrow -\log 2 - \log 2 = -\log 4$

Recognizing KL terms $KL(P\|Q) = \int P\log\frac{P}{Q}$

$C(G) = -\log 4 + KL(P_{data}\|M) + KL(P_g\|M)$

Rewrite JSD $JSD(P\|Q) = \frac{1}{2}KL(P\|M) + \frac{1}{2}KL(Q\|M)$

eq 6 $\Rightarrow C(G) = -\log 4 + 2\,JSD(P_{data}\|P_g)$

Subderivative / subgradient

functions with sharp corners → normal derivative may not exist

A subgradient is a generalized slope that still points downhill

→ In practice we rarely represent a family of P_g distributions via the function $G(z;\theta_g)$ & we optimize θ_g rather than P_g itself

In practice the above assumption is not true

G is a neural network

parameter space is non convex

many critical points exist

Assumptions — 1) infinite / large capacity → G & D can represent needed functions

2) D reaches optimum each step → whenever G changes → D fully optimized

3) small updates to G → so optimization behaves nicely

~~One line summary~~

if we could perfectly optimize D and update G in a convex space

GAN training would converge to real data space

Here $V(G,D) = V(P_g, D)$

treat the objective as a function of generator distr itself

instead of thinking about NN parameters think about distr^ns

Before GAN → how do we learn a complex probability distribution

↳ Deep directed Graphical Models

idea define how data is generated step by step } problems computing probabilities is hard, inference often intractable } training becomes

Explicit probabilistic model

treat the objective as a function of generator distb itself
instead of thinking about NN parameters think about distbns

High level Comparison

	Main Difficulty
Directed Models	difficult inference / likelihood
Undirected models	hard sampling / normalization
Auto encoders	blurry samples / approximation
Adversarial Models	Unstable training → GAN a two player network not sample optimization

GAN advantages relative to others → No Markov chain sampling
→ No explicit likelihood computation
→ Direct sampling via generator network

→ no inference is needed during learning

A wide variety of functions can be incorporated into the model

generator is only updated with gradients from discriminator
that means input components are not copied into generator parameters

Adv nets can even represent very sharp, even degenerate distributions
while markov chains require distribution to be blurry

↳ Deep directed Graphical Models
idea define how data is generated step by step
Explicit probability model
} problems computing probabilities is hard, inference often intractable } training becomes hard

↳ Deep undirected models (Boltzmann machines)
idea Model energy / compatibility instead of directly → sampling required Markov chains, very slow & expensive } training & sampling are hard

↳ Generative Auto encoder (VAE style decoder)
↳ encode data → latent space → decode block } stable training } optimization uses approximations, samples often blurries

GAN avoids explicit probability modelling
Instead learn by competition
(We Dont need to compute likelihood directly)

Oscillations
G finds weakness
D adapts
G moves elsewhere
(Training may increase instead of converging)

Balance problem
if D is too strong
G gets weak gradients, stops learning

Model collapse
if G discovers few samples that fool D well & keeps producing } low diversity

7 Conclusions

Conditional Generative model $P(x|c)$ can be obtained by adding c as input to both G & D } GAN's can be conditioned

Efficiency improvements – training can be accelerated greatly by choosing better methods for coordinating G & D & determining better distributions to sample z from during training

instead of training separate models one can train a shared model that can represent many conditionals { parameter sharing for efficiency }